

READING SHEET

Error Boundaries, Performance & Lazy Loading

Module 3 · Frontend: React & Next.js · Day 34 · Week 7

PROGRAM

CodeOps · Full Stack

WORK TYPE

Revision + mini-project

EST. TIME

2–3 hours

Tip How to use this sheet

This is your take-away for Day 34 — the four concerns that separate a working application from a professional one. Boundaries so a failure stays local, code splitting so the first screen arrives sooner, profiling so any optimisation is earned, and portals so a modal can escape the card that opened it. Everything here applies to the Addis Eats build you have been growing since Day 26.

Part 1 · Error Boundaries

If a component throws during render, React unmounts the **entire** tree — header, cart, checkout and all. The page goes white. That is deliberate rather than a bug: React would rather show nothing than a corrupted interface, because a half-rendered checkout could take a payment for the wrong amount. An error boundary is how you tell it to keep the rest and replace only the broken region.

ErrorBoundary.jsx — the one class you still write

```
class ErrorBoundary extends React.Component {
  state = { error: null };

  static getDerivedStateFromError(error) {
    return { error }; // render the fallback
  }
  componentDidCatch(error, info) {
    logToService(error, info); // report it
  }
  render() {
    if (this.state.error) return this.props.fallback;
    return this.props.children;
  }
}
```

`getDerivedStateFromError` updates state so the fallback renders; `componentDidCatch` is the side-effect half, where you send the error somewhere you will actually read it. There is still no hook for this — in practice most teams install `react-error-boundary` rather than write the class themselves.

What boundaries do not catch

CAUGHT	NOT CAUGHT
Errors during render	Errors inside event handlers
Errors in lifecycle methods	Errors in async code and promises
Errors in a child component	Errors in the boundary itself
Errors in constructors	Errors thrown on the server

Warning Render errors only

A boundary wraps the rendering process, so a click handler, a rejected fetch or a timer never reaches it. Students commonly wrap everything in a boundary and then wonder why a failed request still shows a broken screen — that is what the Day 29 error state is for. The two work together, not instead of each other.

Where to place them

```
App.jsx – boundaries at meaningful seams
<ErrorBoundary fallback={<AppCrashed />}> // last resort
  <Layout>
    <ErrorBoundary fallback={<MenuUnavailable />}>
      <Menu /> // can fail alone
    </ErrorBoundary>
    <ErrorBoundary fallback={<CartUnavailable />}>
      <CartPanel />
    </ErrorBoundary>
  </Layout>
</ErrorBoundary>
```

Ask what should survive if this part breaks. If the menu fails, the cart and header should still work — so the boundary goes around the menu, not the application. And a fallback should say what failed and offer an action: a dead-end apology is only marginally better than the white screen.

Part 2 · Code Splitting & Lazy Loading

By default every screen you have written ships in one bundle, downloaded before anything appears. On a mobile connection every unnecessary hundred kilobytes is a real wait — a student opening the menu should not be paying to download the checkout form and a charting library first.

```
App.jsx – split at the route
import { lazy, Suspense } from "react";

const Checkout = lazy(() => import("../checkout/Checkout"));

<Suspense fallback={<Spinner />}>
  <Routes>
    <Route path="menu" element={<Menu />} />
    <Route path="checkout" element={<Checkout />} />
  </Routes>
</Suspense>
```

lazy takes a function returning a dynamic import. The bundler splits that module into its own file and React downloads it the first time the component actually renders. A Suspense boundary above it is required — without one, a lazy component throws while it loads.

Choosing split points

WORTH SPLITTING	NOT WORTH SPLITTING
Routes other than the landing screen	The first screen people arrive on
Charts, maps, rich text editors	A small component used everywhere
Modals and rarely opened panels	Anything under a few kilobytes
Admin or export screens	Code the first render already needs

Measure before you split: run the production build and look at the output sizes. Splitting a 40 kB application achieves nothing, while splitting one with a heavy chart library achieves a great deal. Each split is also another request and another loading state, so too many small chunks can be slower than one sensible bundle.

Boundaries and Suspense together

```
<ErrorBoundary fallback={<ChartUnavailable />}>
  <Suspense fallback={<ChartSkeleton />}>
    <SalesChart data={orders} />
  </Suspense>
</ErrorBoundary>
```

the pair you will usually write

While the chunk downloads, the Suspense fallback shows. If the download fails — a dropped connection mid-navigation — the import rejects, which is a render error, so the boundary catches it. A **skeleton** roughly the size of what is coming beats a spinner: it stops the layout jumping and makes the wait feel shorter.

Part 3 · Performance in Practice

You met useMemo and useCallback on Day 30 and were told to measure first. This is the measuring. Record a profile of the slow interaction, find the component with the longest bar, ask *why* it rendered, fix that cause, then record again and compare.

TOOL	WHAT IT TELLS YOU
Profiler flamegraph	Which components rendered, and how long each took
Ranked chart	The slowest components in this commit
Highlight updates	What re-renders as you click, live on the page
"Why did this render?"	Which prop or piece of state caused it

Tip Profile something realistic

Development builds are slower than production, and a laptop is far faster than the phone your students will be using. Profile a production build, and throttle the CPU in dev tools if you cannot test on a real device. The component that feels slow is rarely the one that is.

Why components re-render

CAUSE	FIX
Its own state changed	Correct — nothing to do
Its parent re-rendered	React.memo, or move the state down
A prop is a new object each time	useMemo on that value
A handler is recreated each time	useCallback, with a memoised child
The context value changed	Split the context, or use a store

Most unnecessary re-renders come from state living higher than it needs to. If the whole page re-renders because a search box changed, move that state *into* the search box. Structure fixes are free and permanent; memoising is a patch you then have to maintain.

The order to try things

#	STEP
1	Measure — record a profile of the slow interaction
2	Move state down to the component that owns it
3	Split the context, or select narrowly from the store
4	Memoise the specific value or handler that changes
5	React.memo the child that keeps re-rendering
6	Virtualise or paginate if the list is genuinely large

Warning Remove what did not help

The early steps make the code simpler as well as faster; the later ones add complexity and need a measurement behind them, plus a second measurement proving they worked. If a memo made no measurable difference, take it out — unjustified optimisation is code your teammates maintain forever.

Part 4 · Portals

A modal inside a card with overflow: hidden will be clipped, however you style it. So will a tooltip, a dropdown or a toast. CSS cannot fix it — overflow, transform and z-index on an ancestor all trap a child inside it. The problem is where the element lives in the DOM, and a portal is how you move it.

```
Modal.jsx — render somewhere else

import { createPortal } from "react-dom";

function Modal({ children, onClose }) {
  return createPortal(
    <div className="overlay" onClick={onClose}>
      <div className="panel">{children}</div>
    </div>,
    document.getElementById("modal-root")
  );
}

// index.html: <div id="modal-root"></div>
```

Two arguments: what to render, and which DOM node to render it into. The container must already exist — add the target div to index.html outside your app root, or you get a null container and a crash.

Still a child, in every way that matters

A portalled component keeps its place in the **React** tree. It reads the same context it would have read, and its events bubble to its React parent rather than to the DOM node it was rendered into. That is the whole point: you move the pixels without moving the component.

Warning The bubbling surprise

A click inside the modal fires the `onClick` of the card that rendered it, even though the modal is nowhere near it in the DOM. If that is not what you want, stop propagation. This catches out almost everyone the first time.

A modal people can actually use

REQUIREMENT	WHY
Focus moves into the modal	Keyboard users start inside it, not behind it
Focus is trapped while open	Tab must not wander into the page beneath
Escape closes it	The expected way out, on every platform
Focus returns on close	They continue from where they were
<code>role="dialog"</code> and a label	A screen reader announces what opened

The native `<dialog>` element gives you focus handling, Escape and the backdrop for free. Reach for a portal when you need control the element does not offer — not as a reflex.

Assigned Reading**Required**

1. [react.dev — lazy](#) What / why: splitting a component out of the bundle and the Suspense boundary it needs.
2. [react.dev — createPortal](#) What / why: rendering into another DOM node, and the event bubbling rules that surprise people.

Optional — go deeper

- [react-error-boundary](#) — the library most teams use instead of writing the class themselves.
- [React DevTools Profiler](#) — recording a session and reading the flamegraph and ranked chart.

Exercises

Work these in order and commit as you go.

1. Write an `ErrorBoundary` class taking a fallback prop, and wrap the menu in it.
2. Throw deliberately inside one dish and confirm the header and cart still work.
3. Add a second boundary around the cart panel so the two fail independently.
4. Lazy-load the checkout and receipt routes behind a Suspense skeleton.
5. Record a Profiler session while adding three dishes, and note the slowest component.
6. Fix one unnecessary re-render, then record again and compare the two timings.
7. Build a dish modal with `createPortal`, closing on Escape and returning focus on exit.

Mini-Project · Hardening Addis Eats

Tip What you build

Take the routed application and make it resilient and fast. A failure in the menu must not take down the cart. The checkout must not be downloaded by someone who only wants to browse. One re-render must be profiled, fixed and measured again. And the dish modal must escape its card and be usable from the keyboard.

Requirements

- Separate error boundaries around the menu and the cart, each with a useful fallback.
- A deliberate thrown error proving one region fails without taking the others with it.
- The checkout and receipt routes lazy-loaded behind a Suspense skeleton.
- PROFILE.md recording one slow render, what caused it, the fix, and the new timing.
- A dish modal rendered with createPortal, with Escape, focus trapping and focus return.
- No optimisation left in the codebase that you cannot justify with a measurement.

A worked start

```

                                                                    starter – boundary plus lazy route
const Checkout = lazy(() => import("../checkout/Checkout"));

<ErrorBoundary fallback={<MenuUnavailable />}>
  <Suspense fallback={<Skeleton />}>
    <Routes>
      <Route path="menu"      element={<Menu />} />
      <Route path="checkout" element={<Checkout />} />
    </Routes>
  </Suspense>
</ErrorBoundary>

```

What to submit

Your GitHub repository with ErrorBoundary.jsx, the updated App.jsx, ui/Modal.jsx and PROFILE.md showing before and after measurements.

Check yourself

- Does a thrown error in one dish leave the header and cart working?
- Does your fallback tell the person what failed and offer them something to do?
- Open the Network tab — does the checkout chunk download only when you navigate to it?
- Does a failed chunk download show the boundary fallback rather than a blank screen?
- Does PROFILE.md show a real before-and-after difference, not just a guess?
- Can you open, use and close the modal without touching the mouse?